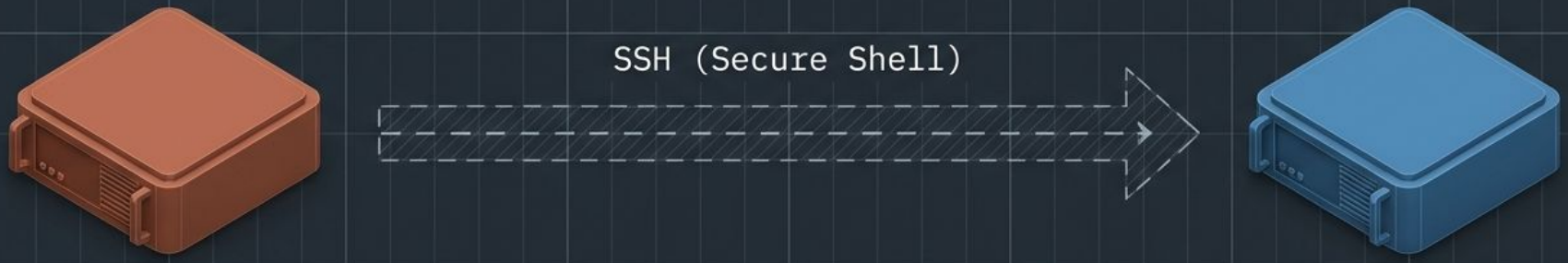
SSH Keys in Linux: A Beginner’s Hands-On Lab

Generate, Verify, and Connect securely without passwords.

Our Mission: Bridge the Network Chasm

Our goal is to securely connect Server 1 to Server 2. We will execute all initial commands from Server 1, forging a secure pathway across the network.



Server 1 (Client)

`10.0.1.10`

Server 2 (Remote)

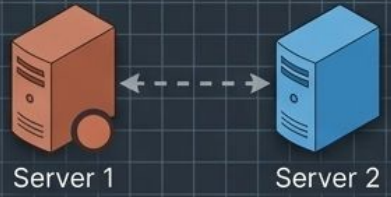
`10.0.1.20`

The Cryptographic Duo: Lock and Key

`id_rsa`	`id_rsa.pub`
Conceptual Role: The Identity / Master Key	**Conceptual Role:** The Lock / Padlock
Physical Location: Stays permanently on Server 1	**Physical Location:** Installed on Server 2
The Golden Rule: KEEP SECRET	**The Golden Rule: CAN BE SHARED**

Pre-Flight Check on Server 1

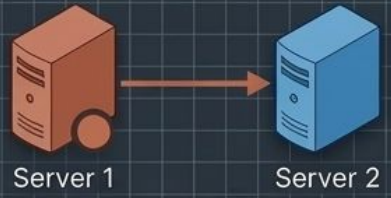
Before we build, we check the foundation. We are verifying our SSH version and checking the hidden .ssh directory to ensure we have a clean slate.




```
$ ssh -V
OpenSSH_9.x

$ ls -la ~/.ssh/
```

Forging the RSA Key Pair




```
$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key: (Press Enter to accept default)
Enter passphrase: (Press Enter for no passphrase in this lab)
```

Inspecting the Forge


```
$ ls -l ~/.ssh/
-rw-------  1 ec2-user ec2-user 2602 id_rsa
-rw-r--r--  1 ec2-user ec2-user  566 id_rsa.pub
```

The creation was successful. We now possess two distinct files. Note the strict permissions (`-rw------`) already applied to the private key.

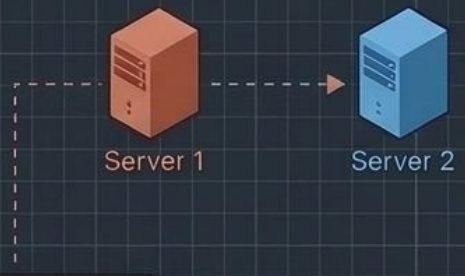
The Private Key: Ultimate Identity


```
$ ssh-keygen -lf ~/.ssh/id_rsa
```

CRITICAL WARNING

You can safely inspect the key type and fingerprint using the command above. However, you must NEVER share the actual contents of id_rsa. It is your master identity. Anyone who holds this file holds the keys to your digital kingdom.

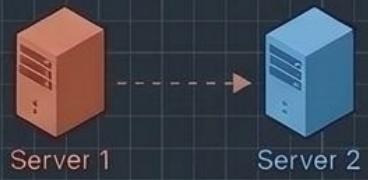
The Public Key: Ready for Transport



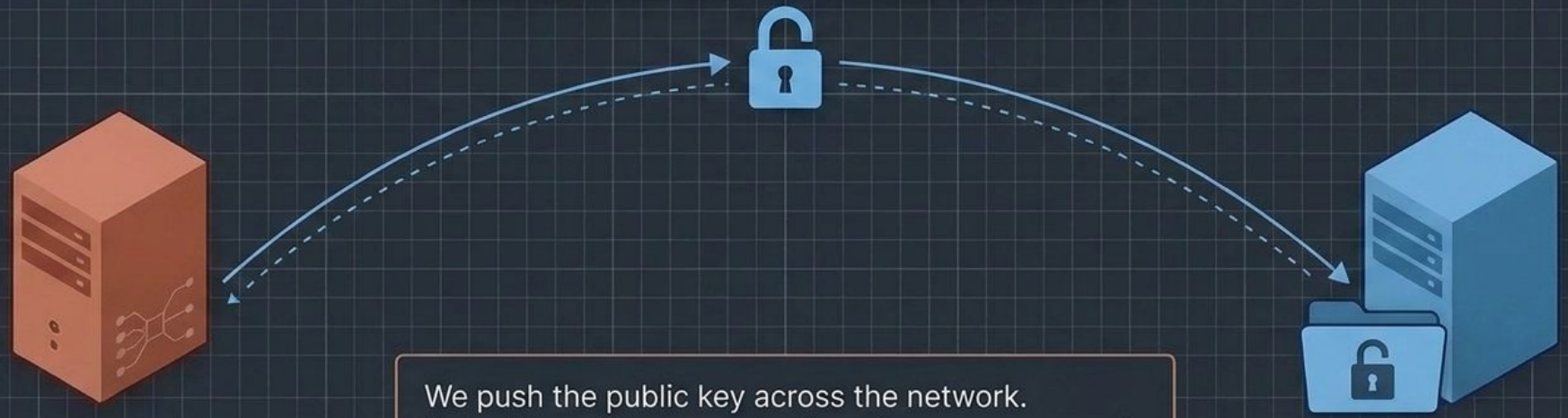

```
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQ...
```

This string of characters is the lock. It is completely harmless on its own. Our next step is to carry this lock across the network and install it on Server 2's door.

Transport and Installation




```
$ ssh-copy-id ec2-user@10.0.1.20
```

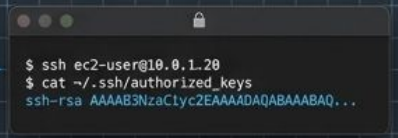


We push the public key across the network.
Note: You will be prompted to enter your password this one final time to complete the installation.

Verifying the Installation on Remote

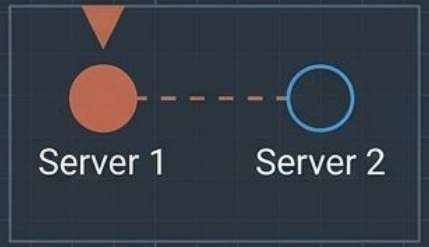

```
$ ssh ec2-user@10.0.1.20
$ cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQ...
```

Text key from Slide 8



The lock is securely in place.

The Seamless Connection

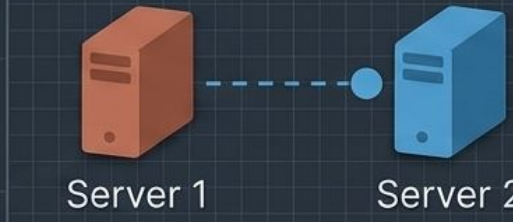



```
$ ssh ec2-user@10.0.1.20
```

SSH automatically locates ~/.ssh/id_rsa.

No password prompt. No friction.
The cryptographic handshake happens invisibly.

Proof of Life



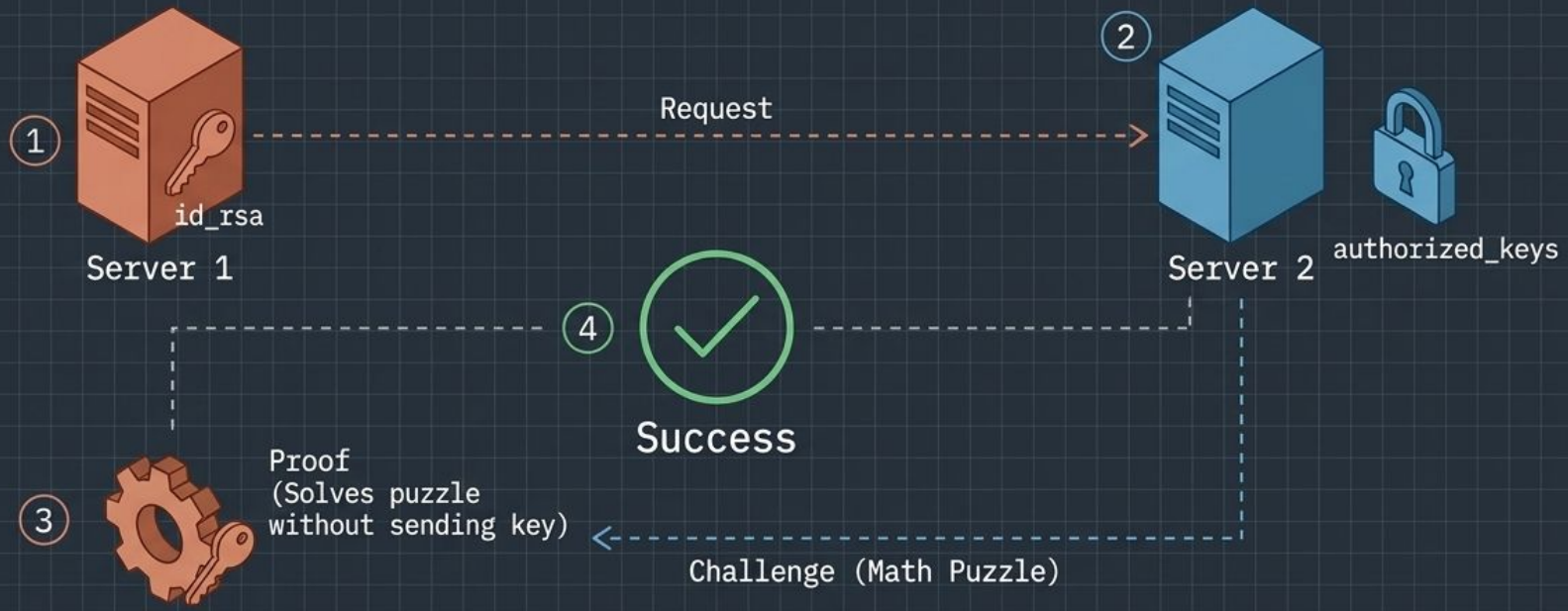

```
$ hostname
server-2
```

```
$ whoami
ec2-user
```

```
$ pwd
/home/ec2-user
```

You are in. Authentication was successful, and these commands verify your new environment.

Synthesis: The Invisible Handshake

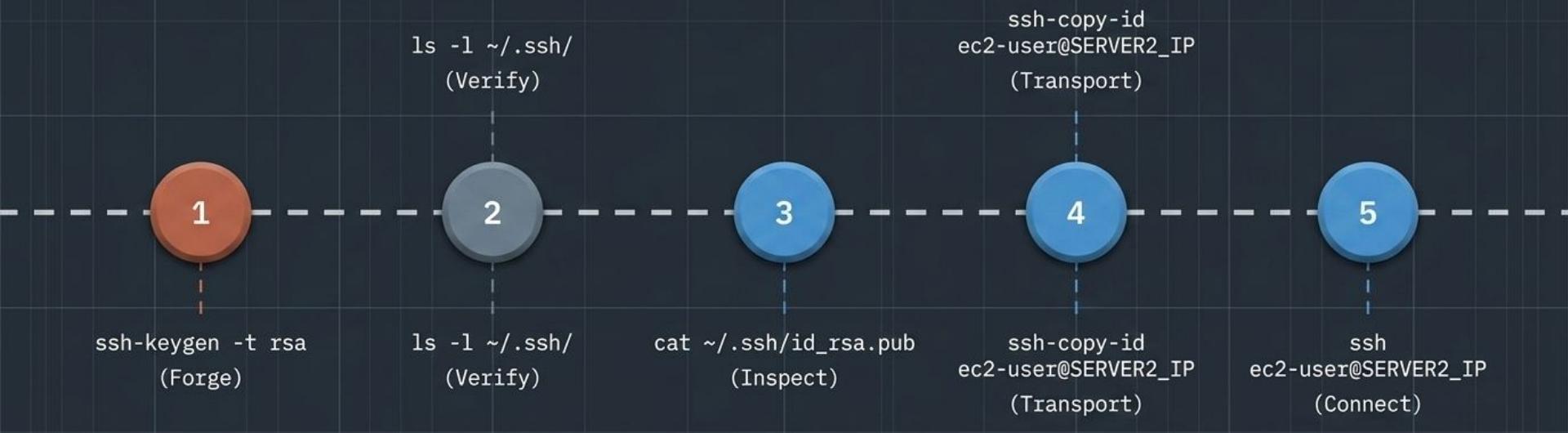


The architecture of passwordless security: The private key never leaves home. The public key stands guard.

Diagnostic Dashboard: Fixing 'Permission Denied'

SSH security is intentionally strict. If your digital doors are left too loose, the system will refuse to protect you.

The Operational Sequence



Remember: Never copy id_rsa to the remote server. Copy only id_rsa.pub.